A Unified Low-Latency ML-KEM Accelerator with Deterministic Packetization and Conflict-Free Memory Mapping

Xiangrui Jia
Tiangong University
Tianjin, China
2431101283@tiangong.edu.cn

Qingzeng Song*
Tiangong University
Tianjin, China
qingzengsong@163.com

Yongjiang Xue
Tiangong university
Tianjin, China
xueyongjiang@tiangong.edu.cn

Weigang Kong
Tiangong University
Tianjin, China
kwg19970502@gmail.com

Fei Qiao
Tsinghua University
Beijing, China
qiaofei@tsinghua.edu.cn

Abstract

National Institute of Standards and Technology (NIST) Federal Information Processing Standards (FIPS) 203 standardizes Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM), driving the demand for low-latency accelerators that handle heterogeneous pipeline rates and complex on-chip memory provisioning. We present a speed-prioritized, unified ML-KEM-512 accelerator with a 16-lane packet interface across Key Generation (KeyGen), Encapsulation (Encaps), and Decapsulation (Decaps). To bridge irregular data streams, an output shaper aggregates accepted coefficients from rejection sampling into deterministic 16-lane blocks, shortening control critical paths. To sustain parallel Number Theoretic Transform (NTT) and Inverse Number Theoretic Transform (INTT) utilization, memory is partitioned by access semantics: sequential boundary traffic uses hierarchical buffers, while the strided transform domain uses a 16-bank store via distributed Look-Up Table Random Access Memory (LUTRAM). XOR bank mapping and stage-aware scheduling eliminate access conflicts inherent in standard mappings, while a Latest Value Table (LVT)-based scheme provides logical Two-Write Two-Read (2W2R) semantics. Implemented on a Xilinx Artix-7 FPGA, the design operates at 225 MHz, completing KeyGen, Encaps, and Decaps in 1131, 1228, and 1641 cycles. Achieving an 17.78 µs lifecycle latency, it demonstrates superior execution efficiency for the complete ML-KEM-512 flow.

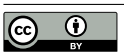
CCS Concepts

• **Hardware → Hardware accelerators**; • **Security and privacy → Hardware-based security protocols**.

Keywords

Post-quantum cryptography; ML-KEM; FPGA accelerator; NTT; conflict-free mapping;

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
GLSVLSI '26, Canandaigua, NY, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2431-2/26/06
https://doi.org/10.1145/3787109.3815252

ACM Reference Format:
Xiangrui Jia, Qingzeng Song, Yongjiang Xue, Weigang Kong, and Fei Qiao. 2026. A Unified Low-Latency ML-KEM Accelerator with Deterministic Packetization and Conflict-Free Memory Mapping. In *Great Lakes Symposium on VLSI 2026 (GLSVLSI '26), June 22–24, 2026, Canandaigua, NY, USA.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3787109.3815252

1 Introduction

Large-scale quantum computing threatens widely deployed public-key cryptosystems (e.g., Rivest-Shamir-Adleman(RSA) [15] and Elliptic Curve Cryptography(ECC) [10, 11]) via Shor's algorithm [16], motivating the shift to post-quantum cryptography. Following NIST's post-quantum standardization (finalized in 2024), FIPS 203 [13] specifies Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM, formerly CRYSTALS-Kyber), where low latency and implementation efficiency are critical for hardware accelerators, since key establishment often lies on the critical path of secure-session setup. In high-throughput networks, relying solely on software execution introduces unacceptable latency overheads, making dedicated hardware acceleration an absolute necessity.

However, acceleration of ML-KEM cannot be achieved simply by making the arithmetic core faster. The execution pipeline is inherently heterogeneous across stages. Sampling for generating the public matrix A exhibits irregular valid-output cadence, whereas later polynomial arithmetic typically operates on fixed-size coefficient groups under a fixed parallel datapath. If accept/reject decisions, packet assembly, and writeback are tightly coupled, control paths become deep and timing closure becomes difficult. Meanwhile, the throughput bottleneck is often the banked polynomial store that serves strided, high-parallel accesses in the transform path (Number Theoretic Transform and Inverse Number Theoretic Transform(NTT/INTT)). Unlike arithmetic units, the memory subsystem must simultaneously satisfy bandwidth, access regularity, and conflict-free provisioning constraints. Each cycle must provision multiple coefficient groups to the compute array in parallel, while writing back results from earlier batches, and bank conflicts in the resulting memory accesses stall the compute array and waste bandwidth. For this reason, high-speed NTT hardware commonly emphasizes conflict-avoiding mappings and access schedules to prevent stalls.Therefore, achieving optimal hardware performance requires meticulously harmonizing the computational datapath with a highly deterministic memory provisioning system.

To address the issues above, we make the following contributions:

- We implement a unified ML-KEM-512 accelerator supporting KeyGen, Encaps, and Decaps; on a Xilinx Artix-7, the design achieves 225 MHz and completes the full lifecycle in 17.78 μs, utilizing 16-lane deterministic packetization and XOR mapping principles that are highly scalable to higher security levels.
- We introduce rejection-sampling output shaping and time slicing, converting irregular valid-coefficient sequences into fixed 16-lane packets, standardizing inter-stage interfaces and simplifying scheduling.
- We separate sequential/bursty stage-boundary traffic into buffer/FIFO layers and reserve the banked polynomial store for NTT/INTT. We apply XOR-based conflict-free mapping and an LVT-based dual-replica scheme to provide read-latest semantics with logical 2W2R decoupling, aligning a 16-lane datapath with 16-bank NTT provisioning.

2 Background and Related Work

2.1 Algorithmic Bottlenecks and Data Provisioning

According to FIPS 203 [13], the ML-KEM hardware execution flow comprises three datapaths with highly heterogeneous memory behaviors:

1) **Polynomial arithmetic:** NTT/INTT exhibits large-stride, partially irregular access patterns. As highly parallel butterfly units (based on standard Cooley-Tukey or Gentleman-Sande principles) fetch multiple coefficients concurrently, power-of-two strides inevitably cause structural bank conflicts under linear memory mappings.
2) **Conversion and compression:** This datapath is predominantly sequential but exhibits bursty reads/writes at object boundaries due to bit-level packing.
3) **Randomness and derivation:** XOFs expand seeds into pseudorandom streams, shaped by rejection sampling or centered binomial distribution(CBD). The irregular valid-coefficient cadence caused by rejection sampling is a well-known hardware bottleneck across various post-quantum cryptography families (e.g., Multivariate Public Key Cryptography (MPKC) [8]). If these irregular outputs are not properly regularized, they cause timing violations or structural stalls in parallel memory architectures.

Mapping these mixed-semantics accesses onto the same banked memory fabric forces the interconnect to support parallel multi-lane provisioning and sequential byte-stream movement simultaneously. Frequent switching introduces alignment overhead and arbitration bubbles, motivating our semantics-partitioned storage organization.

2.2 Related Work

Recent ML-KEM/Kyber hardware increasingly emphasizes that performance is dictated not only by butterfly arithmetic, but also by data provisioning—i.e., whether the memory system can deliver conflict-free, cycle-regular coefficient groups under high parallelism. Several prior works implement unified accelerator architectures that cover the full ML-KEM lifecycle (KeyGen, Encaps, and Decaps) through module reuse and top-level scheduling. Prior work [18] presents a compact FPGA implementation of CCA-secure CRYSTALS-Kyber[1] via arithmetic-unit reuse and FIFO-based scheduling; it achieves low resource utilization, but is ill-suited for latency-sensitive scenarios. Another line of work [9] supports all three security levels ($k \in \{2, 3, 4\}$) within a single configurable instance; however, general configurability limits specialization for a fixed k, leaving room for further latency and throughput optimization.

To mitigate bank conflicts in parallel NTT/INTT, prior works propose conflict-avoidance access schedules and bank-mapping schemes to improve effective parallel-port utilization, including Kyber-specific access patterns and mixed-radix/split-radix NTT designs [6, 7], as well as scalable radix-2/4 conflict-free mappings [2]. More generally, methodology-driven studies formalize scalable conflict-free NTT accelerator design and establish correctness and provisioning conditions for parallel memory access [12]. However, these methods typically focus on conflict-free fetch/writeback for the NTT kernel and assume the banked fabric is provisioned for strided/irregular access. Co-locating sequential, block-oriented traffic (e.g., pointwise ops, encoding, bulk movement) in the same banked domain increases arbitration/merge and alignment logic; scaling parallelism often requires more banks and interconnect fanout, worsening routing and timing closure.

Beyond memory-based NTT implementations, structured pipelines such as MDC architectures use delay lines and switching networks to sustain high throughput with a regular cadence [3, 14], but introduce priming latency and limit cross-operation reuse. In parallel, kernel-focused NTT/INTT accelerators aggressively reduce NTT latency (e.g., 72 cycles [4] and even single-digit cycles [17]). Yet once NTT is no longer dominant, bottlenecks migrate to hashing, sampling, and pointwise stages, while higher parallelism amplifies provisioning pressure such that coefficient storage, bank scaling, and interconnect routing become critical constraints for overall throughput and timing closure.

Recent TLP architectures [5] reduce cycles through dynamic arbitration, but runtime collision resolution introduces combinational overhead. In contrast, our purely deterministic provisioning system prioritizes clock frequency and absolute latency. By adopting a semantics-partitioned storage that isolates strided hot-spot traffic from boundary streaming, we eliminate the interconnect overhead of unified banked fabrics. Together with deterministic scheduling, this ensures high frequency and tight timing closure across the full ML-KEM lifecycle.

3 System Architecture Design

3.1 Top-Level Architecture

Figure 1 illustrates the top-level architecture of the proposed unified ML-KEM-512 accelerator. To achieve absolute minimal wall-clock latency, the design is centered around a highly parallel 16-lane datapath, capable of processing 16 coefficients per clock cycle. As depicted, the system is orchestrated by a top-level Control Unit

and comprises specialized hardware modules: the Sampling & Hash Unit, the Conversion and Compression Unit, and three dedicated arithmetic cores (NTT/INTT, Pointwise mul engine, and add/sub engine).

While this paper focuses on demonstrating the absolute minimal full-lifecycle latency for ML-KEM-512, the proposed architecture is intrinsically extensible. The 16-lane deterministic packetization interface, the compute datapath, and the XOR conflict-free memory mapping are fully parameterizable. Extending support to ML-KEM-768 or ML-KEM-1024 primarily requires adjusting the Base_Offset capacity boundaries and algorithmic loop bounds within the control unit, without necessitating any structural modifications to the core 16-bank logical 2W2R LUTRAM provisioning system.

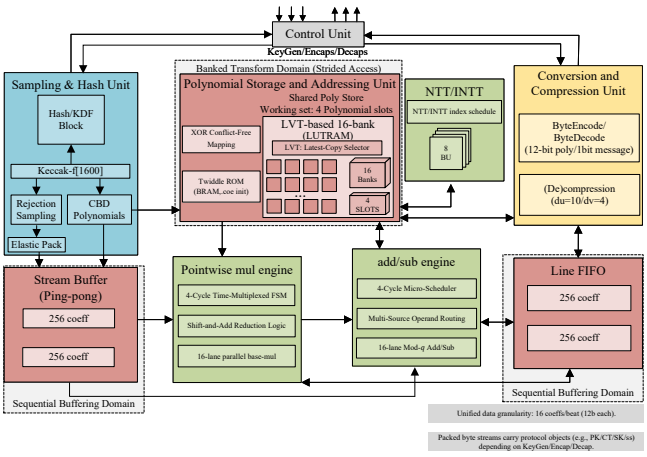


Figure 1: Top-level architecture of the proposed unified 16-lane ML-KEM-512 accelerator.

Rather than relying on a centralized memory bottleneck, the on-chip storage is explicitly decoupled based on access semantics. The Polynomial Storage and Addressing Unit features a distributed LVT-based 16-bank LUTRAM domain to accommodate the highly parallel, strided accesses required by polynomial transforms. In contrast, sequential boundary traffic is isolated into two complementary buffering structures:

1) **Stream Buffer (Ping-pong):** Decouples the production–consumption path of sampling. The hash unit writes blocks sequentially into one page, while the downstream consumer reads from the other. Synchronization and backpressure are applied only at page switches, confining waiting to block boundaries.
2) **Lane FIFO:** Temporarily stores intermediate results and aligns arrival timing. It serves as the primary sequential bridge for the Conversion and Compression Unit and buffers intermediate sequential results from the pointwise multiplication.

On top of the unified 16-lane granularity and partitioned storage, the Control Unit orchestrates the three protocol flows (KeyGen, Encaps, and Decaps) using a mode abstraction. Flow switching is realized by reconfiguring stage sequencing and base-offset selection rather than by duplicating datapaths. This enables seamless stage overlap and fast mode switching without structural stalls.

The selection of a 16-lane parallelism ($P = 16$) is a strategic decision centered on the physical resource constraints and the dataflow characteristics of ML-KEM. While a lower parallelism (e.g., $P = 8$) would reduce the compute datapath area, it fails to satisfy the stringent real-time requirements, effectively doubling the execution cycles of the compute core. Conversely, scaling to a 32-lane configuration ($P = 32$) necessitates a 384-bit wide internal data bus, which induces severe combinatorial routing congestion and high-fanout delays in the address-generation logic. More importantly, the speedup achieved by increasing compute parallelism is fundamentally non-linear due to the throughput bottleneck of the symmetric engine. Since the Keccak-based hashing and rejection sampling units generate valid coefficients at a constrained cadence, an oversized 32-lane compute core would spend significant intervals under-utilized while waiting for the randomness generation path to replenish the data stream.

3.2 Banked Polynomial Storage and Address Mapping

To match the 16-way compute array, the polynomial storage is partitioned into $B = 16$ logical banks. To avoid capacity fragmentation under shallow bank depths, we implement this fabric using distributed memory (LUTRAM). We introduce a Latest Value Table (LVT) mechanism to provide logical 2W2R semantics, enabling read/write decoupling.

To efficiently accommodate multiple sets of polynomials across the KeyGen, Encaps, and Decaps flows within the same physical array, the design introduces a parameterized slot management mechanism. We partition the physical array depth into four logical slots, where each slot can hold one full polynomial of $N = 256$ coefficients. A base offset (Base_Offset) issued by the top-level control logic participates in the computation of the physical row address:

$$\text{Phy_Row} = (\text{Log_Row} \bmod \text{Slot_Depth}) + \text{Base_Offset} \quad (1)$$

This mechanism ensures the physical residency and isolation of multi-task data. The base offset acts as a slot-level pointer used to place different polynomials into disjoint row ranges within the same physical bank array. Selecting a different base offset effectively selects a different polynomial-residency window, allowing seamless transitions between protocol phases without explicit data movement.

A core bottleneck in NTT is that the memory-access stride changes exponentially. When the stride is a power of two (2^k) and shares a common divisor with the bank count $B = 16$, standard linear interleaving maps parallel indices to the same bank, causing congruence clustering. To eliminate this, we apply an XOR-based conflict-avoidance mapping:

$$\text{Bank_ID} = (\text{idx} \bmod P) \oplus \lfloor \text{idx}/P \rfloor, \quad \text{LocalAddr} = \lfloor \text{idx}/P \rfloor. \quad (2)$$

The XOR operation mathematically breaks this modulo-congruence by scrambling the higher-order group bits into the bank ID, ensuring a bijective (one-to-one) mapping between the 16 parallel lanes and the 16 physical banks across all NTT stages.

Stage dependence is handled by the butterfly index generator: given the stage and group ID, it emits the logical index sequence, which the XOR mapping deterministically translates into physical locations without structural bubbles (Fig. 2).

Mathematically, this XOR-based mapping guarantees conflict-free accesses because the NTT strides are strictly powers of two

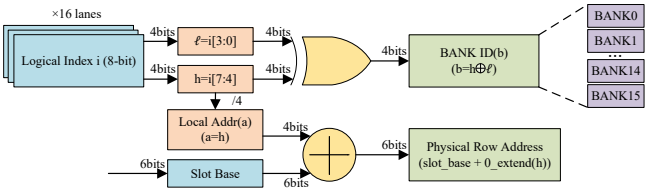


Figure 2: XOR bank mapping and slot-based physical row addressing for the 16-bank polynomial store.

(2^s). In any single clock cycle, the 16 concurrently requested indices $\{idx_0, idx_1, \ldots, idx_{15}\}$ are algebraically derived by toggling specific bit positions of a base address. Since the XOR operator ($\oplus$) acts as a linear bijection over the Galois field GF(2), mixing the lower 4 bits (intra-group index) with the higher bits (inter-group address) ensures that no two distinct indices in the same parallel request evaluate to the same 4-bit Bank_ID. This mechanism provides a strict mathematical permutation across the 16 banks for any power-of-two stride, strictly eliminating structural contention without the high area overhead of prime-factor modulo arithmetic.

3.3 Compute Datapath and Micro-Scheduling

The compute datapath abstracts NTT/INTT and base-field arithmetic (pointwise multiplication and add/sub) as streaming processing over 16-lane groups. We reuse the same butterfly network for both transforms by reconfiguring the internal dataflow: NTT strictly follows a "Multiply → Add/Subtract" sequence, whereas INTT performs "Add/Subtract → divide-by-2 → Multiply", keeping both paths latency-matched (Fig. 3).

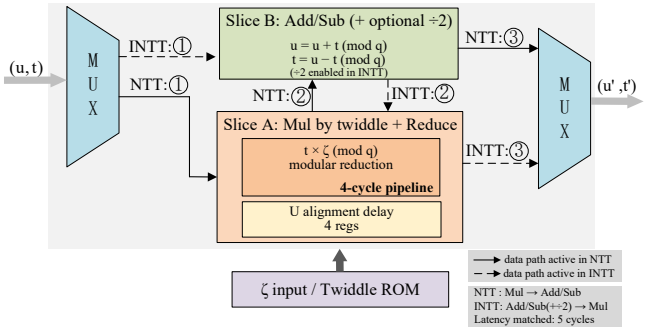


Figure 3: Reused NTT/INTT butterfly pipeline with stage scheduling and optional division by 2.

To avoid explicit inter-stage permutations in NTT/INTT, the index generator uses next-stage-driven lookahead (Fig. 4) to match the next stage's input-demand order. For example, when computing stage 7, if the subsequent stage 6 first consumes coefficients at indices 0 and 64, we schedule the production and writeback of index 64 early enough to be ready before that specific read. This strategy effectively eliminates read-after-write hazards and pipeline bubbles under banked in-place updates.

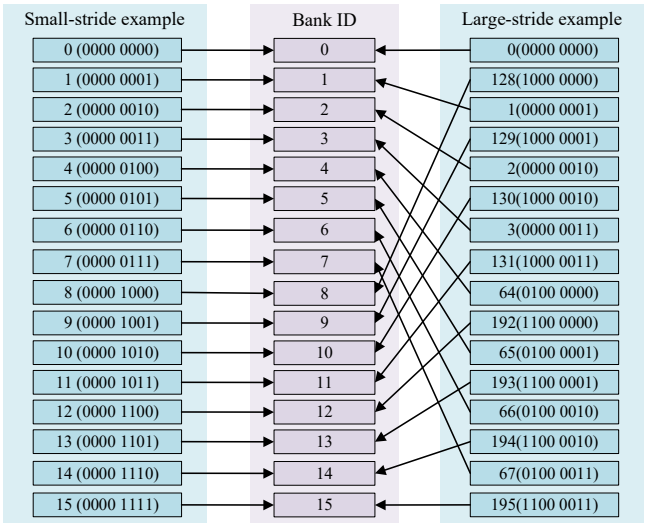


Figure 4: Stage-aware index scheduling under XOR mapping (small-stride example: stage 0, group 1; large-stride example: stage 7, group 1).

Regarding base-field operations, operands are dynamically sourced from the semantics-partitioned domains. Both pointwise multiplication and add/sub follow a strict four-cycle-per-group issuance. Within this window, the first two cycles are dedicated to fetching two 16-lane input operands. We adopt this cadence for multiplication because it is not the primary throughput bottleneck in the protocol flow; thus, the 16-way parallel products can be time-multiplexed across four cycles, aggressively reusing DSP resources. Furthermore, the add/sub engine uses the same four-cycle grouping to stay strictly phase-aligned with multiplication. This enables the direct bypass of multiplication results into the add/sub pipeline at group boundaries, entirely eliminating extra retiming registers or intermediate memory writebacks.

Twiddle factors for butterfly and NTT-domain pointwise multiplication are stored in a ROM packed to serve one parallel group per cycle. Each cycle the ROM outputs a pre-packed wide word that is unpacked and fed to the parallel units, keeping twiddle supply off the critical path. NTT/INTT reuse the same ROM via forward/inverse table selection (with scan direction), while other modes select the corresponding factor set through a starting offset under sequential addressing.

All operators follow a common micro-scheduling template (*address generation → load → compute → writeback*). The pipeline configuration supports continuous group processing, facilitating the interleaved execution of compute and memory tasks.

3.4 XOF/Hash, Sampling, and Output Shaping

Since rejection-sampling accept events are stochastic, directly writing accepted coefficients into the banked memory causes severe routing contention. We resolve this by applying cadence-aligned "time slicing" with an elastic accumulate-and-pack stage (Fig. 5). This "aggregate-packetize-writeback" paradigm converts an irregular valid sequence into a deterministic 16-lane packet stream.

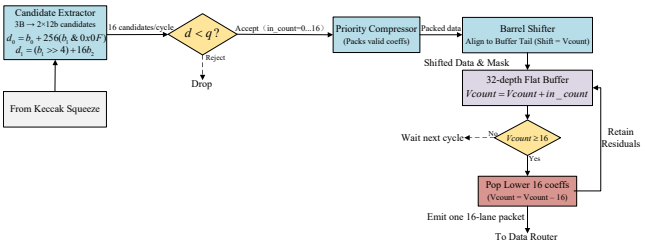


Figure 5: Rejection-sampling output shaper: $d < q$ acceptance filtering, time-sliced accumulation, and fixed 16-lane packet emission.

The generation of secret-dependent noise polynomials (s, e, and y) relies on the CBD sampler, which inherently consumes a fixed, deterministic number of squeezed bits per coefficient. Our hardware datapath strictly preserves this property: the CBD sampling path operates in rigorous constant-time, completely bypassing the elastic accumulation stage.

Crucially, these outputs are routed strictly by access semantics: polynomials destined for NTT/INTT are written to the banked storage domain, while coefficient streams bypassing the transforms are forwarded through the sequential buffering domain. Exceptions are made for specific sequential data (e.g., noise e_2 and decoded message m), which are allocated to the banked domain to prevent sequential buffer bottlenecks.

3.5 Conversion and Compression

The encoding and compression subsystem implements the ML-KEM-specified ByteEncode/ByteDecode and Compress/Decompress functions, converting between 12-bit coefficient arrays and external byte streams via bit-level slicing and alignment. It processes data at a fixed 16-lane granularity to match the unified system cadence.

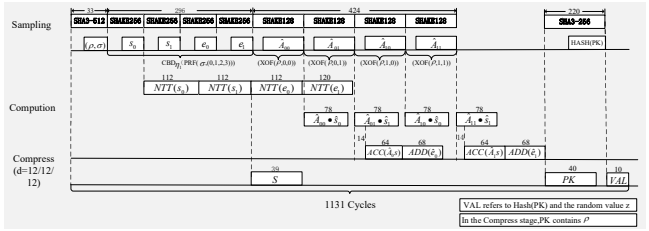
Data routing strictly depends on access semantics. For vectors bypassing NTT/INTT, conversion traffic streams entirely through the line-buffer queue (Line-FIFO), completely offloading sequential operations from the heavily contended banked domain. Conversely, when coupled with transform computations, the subsystem interacts directly with the banked random-access domain. Specifically, KeyGen ($\hat{s}$) and Encaps (post-INTT) read directly from the banks for encoding and compression, while Decaps sequentially loads decoded ciphertext into the banks to prepare for INTT.

4 Implementation and Evaluation

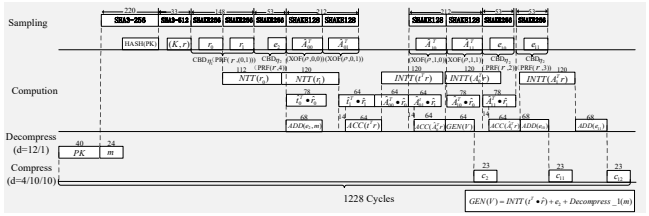
4.1 Implementation and Verification

The proposed design is implemented on a Xilinx Artix-7 XC7A-100T device using Vivado 2021.2. To avoid long combinational paths across wide control networks, we insert pipeline registers and adopt lockstep alignment. The design achieves a 227 MHz post-route $F_{\max}$ and is operated safely at 225 MHz.

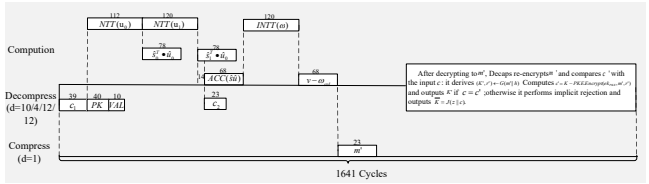
The functional correctness and protocol-level robustness are rigorously validated via a high-speed hardware-in-the-loop (HIL) testing platform utilizing a Gigabit Ethernet interface. We implement a bidirectional role-alternating model to fully exercise the ML-KEM lifecycle. In the Server-side verification flow, the FPGA initiates the transaction by executing KeyGen to generate the keys, which are then streamed to a host PC. Upon receiving the resulting ciphertext from the host, the FPGA performs Decaps to recover the shared secret. Conversely, in the Client-side flow, the FPGA receives a public key and executes Encaps to generate the ciphertext. This dual-role framework ensures that our deterministic packetized interface correctly handles protocol objects under real-world I/O constraints. Across 10,000 closed-loop hardware tests for each security level, the shared secrets strictly match the software golden references, confirming the architectural integrity across the entire FIPS 203 parameter space.



(a) KeyGen



(b) Encaps



(c) Decaps

Figure 6: Cycle breakdown of (a) KeyGen, (b) Encaps, and (c) Decaps.

4.2 Performance and Cycle Breakdown

Figure 6 illustrates the detailed execution timing and stage overlap for the full ML-KEM-512 lifecycle. The three horizontal lanes map to (i) Keccak/XOF generation with on-chip sampling/shaping, (ii) polynomial arithmetic (NTT, pointwise MAC, add/sub), and (iii) conversion/compression between coefficient arrays and protocol byte strings. In the "(De)Compress" lane, $d = 10/4$ denote ciphertext compression precisions ($\text{Compress}_{10}(u)$ and $\text{Compress}_4(v)$), and $d = 1$ denotes the 1-bit channel for m/m' encoding/recovery. For interface-granularity and boundary alignment, we also place the $d = 12$ path in the "Compress" lane, which is equivalent to 12-bit pack/unpack on valid Z_q coefficients.

Once the pipeline is formed, the execution windows for the core NTT/INTT, pointwise multiply, and add/sub stages strictly take 112, 64, and 64 cycles, respectively. By organizing heterogeneous operators into the semantics-partitioned storage domains discussed in Section 3, continuous group processing enables deep stage overlap. As depicted in the timing diagram, this overlap effectively hides

the memory alignment and arbitration latency entirely under the compute cadence.

Furthermore, a critical scheduling optimization is applied during Encapsulation and the re-encryption step of Decapsulation: the top-level scheduler strictly dictates computing the polynomial v prior to u. Since the polynomials decoded from the public key temporarily reside in the line-FIFO (sequential domain), computing v early rapidly consumes these resident data. This naturally frees up the sequential FIFO space required for the subsequent computation of u, thereby preventing structural buffer congestion without artificially increasing the physical depth of the on-chip FIFO.

Lastly, regarding the rejection sampling process, although the number of underlying Keccak permutations is theoretically unbounded due to the stochastic nature of rejection events (requiring at least 3 Keccak blocks to generate enough bytes for one polynomial), our empirical profiling reveals that almost all polynomials are successfully sampled within exactly 3 Keccak operations. This tight empirical bound guarantees that the hashing and sampling phases maintain a highly deterministic execution window, preventing unpredictable timing variations in the overall lifecycle.

4.3 Ablation Analysis

To isolate the contribution of output shaping, we construct Abl-1 by replacing the proposed shaped-output interface with an immediate-writeback scheme. In this configuration, each cycle produces 16 rejection-sampling candidates and, after filtering (d<q), yields a variable number of valid coefficients (0–16). Because the valid count is not fixed and the accepted positions are sparse, these coefficients can no longer be forwarded as a contiguous fixed-width 16-lane packet; instead, they must be written back directly into the finer-grained banked random-access domain. This change breaks the fixed-packet interface assumed by the downstream provisioning logic and invalidates the full-group parallel-access assumption under which the XOR-based bank mapping is conflict-free. As a result, two sparse accepted indices in the same cycle may map to the same bank, thereby triggering same-cycle write conflicts. For example, if idx=63 and idx=72 are accepted in the same cycle, both map to bank 12 under the XOR-based rule, causing a same-cycle write conflict. Preserving correctness then requires additional arbitration, time-multiplexing, or inserted stalls, which increases control/routing fanout, lengthens critical paths, and raises LUT cost.

Table 2: Post-route cost of var-count WB versus this work

Config	Fmax(MHz)	LUT	FF	BRAM
This work	225	23826	14885	8.5
Abl-1	73	29676	17645	8.5

Table 2 summarizes the post-route implementation cost of Abl-1. Removing output shaping reduces the achievable frequency from 225 MHz to 73 MHz while increasing LUT and FF usage, showing that the main benefit of output shaping is not merely interface regularization, but preserving timing closure by avoiding irregular sparse writeback into the banked random-access domain.

To isolate the contribution of the bank-mapping strategy, we construct Abl-2 by replacing the XOR conflict-avoidance mapping with sequential linear interleaving, i.e., mapping bank/address in logical index order. Under this mapping, we generate the standard in-place Cooley–Tukey access sequence for the ML-KEM NTT over 7 stages (len=128→2), and perform a structural conflict simulation assuming 16 coefficient indices issued per cycle with 1R per bank. The results show 112 issued cycles across the 7 stages, with 64 conflict cycles and a worst per-cycle bank multiplicity of 2, implying a lower bound of 64 extra waiting cycles and a total lower bound of 176 cycles. Conflicts concentrate in the first four stages (len=128/64/32/16), because when len is a multiple of 16, (j, j+len) maps to the same bank under sequential interleaving, creating unavoidable structural contention under parallel provisioning.

Note that Abl-1 is evaluated after synthesis and place-and-route, whereas Abl-2 reports a structural conflict simulation under the same 16-lane NTT issue model; the two ablations target different bottlenecks and therefore use different metrics. These results show that output shaping preserves timing closure by regularizing irregular rejection-sampling outputs, while XOR-based bank mapping preserves NTT provisioning efficiency by eliminating structural bank conflicts under high-parallel strided accesses.

4.4 Comparison

To position this work against prior ML-KEM/Kyber hardware implementations, we follow the reporting practice commonly adopted in the literature: resource utilization and clock frequency are taken from post-route implementation reports, while the system-level latency is reported using the complete KeyGen/Encaps/Decaps transaction cycle counts.

Beyond the basic metrics (LUT/FF/DSP/BRAM and frequency), we further include two composite metrics—the Equivalent Number of Slices (ENS) and the Area-Time Product (ATP)—to assess the area-performance trade-off under heterogeneous resource mixes. ATP is defined as the product of latency and ENS:

$$\mathrm{ATP} = \mathrm{Latency} \times \mathrm{ENS}. \tag{3}$$

Here, ENS converts heterogeneous resources into a slice-equivalent proxy:

$$\mathrm{ENS} = 100 \cdot \mathrm{DSP} + 196 \cdot \mathrm{BRAM} + \frac{\mathrm{LUT}}{4} + \frac{\mathrm{FF}}{2}. \tag{4}$$

We compute latency from total cycle count and post-route frequency, and BRAM is reported in 36Kb-equivalent units. These definitions establish a normalized baseline, enabling a mathematically consistent comparison across designs that trade off "faster-but-larger" versus "smaller-but-slower."

Although our architecture incurs a higher ATP of 0.35 and LUT utilization, this is a deliberate trade-off. The ENS inflation stems from using distributed LUTRAMs for the 16-bank polynomial store. Combined with our XOR mapping, this storage provides fully parallel memory access, eliminating bank conflicts and pipeline bubbles to achieve an exceptionally low lifecycle latency of 17.78 μs.

As indicated by the ATP metric, this design aggressively trades hardware area for minimal execution latency. This is a deliberate architectural choice. Our increased resource utilization stems directly from heavily overprovisioning distributed LUTRAMs to strictly eliminate bank conflicts and fully unrolling the compute datapath with 46 DSPs to match the massive 16-bank bandwidth. By making

Table 1: Comparison of hardware resources and performance on Artix-7 platform (ML-KEM-512).

Design	LUT / FF	DSP	BRAM	f (MHz)	Cycles (KG/EN/DE)	Delay μs (KG/EN/DE)	Throughput (kOps/s) Cli./Srv.	ENS	ATP (10^{-3})	Device
[9]	25757 / 18629	21	14	210	1800 / 1900 / 3200	8.57 / 9.05 / 15.24	42.0 / 110.5	20598	0.68	–
[14]	13999 / 12503	2	0	208	1100 / 1500 / 2100	5.30 / 7.20 / 10.10	64.9 / 138.9	9951	0.22	XC7A200T
[6]	7300 / 3200	5	4	200	2400 / 3000 / 4200	12.1 / 15.0 / 21.2	30.0 / 66.7	4709	0.23	XC7A100T
[1]	6910 / 3270	3	3	200	2960 / 3910 / 5230	14.3 / 21.8 / 30.4	22.4 / 45.9	4251	0.28	XC7A100T
[3]	11074 / 10132	10	8	225	1403 / 1754 / 2394	6.23 / 7.79 / 10.64	59.3 / 128.4	10403	0.24	XC7A35T
[5]	13824 / 9631	16	12	210	938 / 1090 / 1936	4.47 / 5.19 / 9.22	73.0 / 192.7	12224	0.23	–
This work	**23826 / 14885**	**46**	**8.5**	**225**	**1131 / 1228 / 1641**	**5.03 / 5.46 / 7.29**	**81.2 / 183.1**	**19665**	**0.35**	**XC7A100T**

Note: All implementations are evaluated on the Xilinx Artix-7 FPGA platform. KG/EN/DE denotes KeyGen, Encaps, and Decaps phases respectively. For throughput, we assume an ephemeral key exchange model (Client=KeyGen+Decaps; Server=Encaps).

this targeted area investment, we completely removed structural stalls and complex arbitration logic.

To demonstrate the system-level superiority of this trade-off, we analyze the absolute throughput in a standard ephemeral key exchange model, where the Client executes KeyGen and Decapsulation, while the Server performs Encapsulation. Based on our exact execution delays, the proposed design achieves an industry-leading Client-side throughput of 81.2×10^3 operations per second (ops/s) and a highly competitive Server-side throughput of 183.1×10^3 ops/s. For strictly latency-bound cryptographic applications where absolute wall-clock latency acts as a hard functional deadline, trading silicon area for extreme protocol throughput is highly justified, allowing our architecture to reliably break the 18 μs execution barrier.

5 Conclusion

This work presents a unified 16-lane ML-KEM-512 accelerator optimized for minimal wall-clock latency. By integrating rejection-sampling output shaping with semantics-partitioned storage and XOR-based bank mapping, the architecture achieves deterministic cross-stage provisioning and eliminates structural arbitration stalls. Implemented on a Xilinx Artix-7 FPGA, the design operates at 225 MHz and completes the full KeyGen-Encaps-Decaps lifecycle in 17.78 µs. This establishes a new state-of-the-art by reliably breaking the 18-microsecond execution barrier among full-lifecycle designs on this platform. Future work will extend this deterministic framework to higher security levels and integrate side-channel countermeasures.

Acknowledgments

This work was supported by the Tianjin Major Special Project for Artificial Intelligence Science and Technology under Grant No. 25ZXRGGX00100.

References

[1] Joppe Bos, Leo Ducas, Eike Kiltz, T Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehle. 2018. CRYSTALS - Kyber: A CCA-Secure Module-Lattice-Based KEM. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*. 353–367. doi:10.1109/EuroSP.2018.00032

[2] Xiangren Chen, Bohan Yang, Shouyi Yin, Shaojun Wei, and Leibo Liu. 2021. CFNTT: Scalable Radix-2/4 NTT Multiplication Architecture with an Efficient Conflict-free Memory Mapping Scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2022, 1 (Nov. 2021), 94–126. doi:10.46586/tches.v2022.i1.94-126

[3] Song Cheng, Jiansheng Chen, Jianyang Li, Kan Yao, Shunxian Gao, Kangkang Rui, and Yijun Cui. 2025. Optimized Design and Implementation of CRYSTALS-KYBER Based on MLWE. *Security and Communication Networks* 2025, Article 7884158 (2025), 15 pages. doi:10.1155/sec/7884158

[4] Dixit Dutt Bohra, Dip Sankar Banerjee, and Somitra Sanadhya. 2025. Sparse-Aware NTT: Accelerating Lattice-Based Cryptography on FPGAs. In *2025 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, Vol. 1. 1–6. doi:10.1109/ISVLSI65124.2025.11130302

[5] Peihao Fan, Ziqin Yan, Wu Guan, and Liping Liang. 2026. A High-Performance ML-KEM Architecture With Task-Level Parallelism. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* (2026), 1–14. doi:10.1109/TVLSI.2026.3677808

[6] Wenbo Guo and Shuguo Li. 2023. Highly-Efficient Hardware Architecture for CRYSTALS-Kyber With a Novel Conflict-Free Memory Access Pattern. *IEEE Transactions on Circuits and Systems I: Regular Papers* 70, 11 (2023), 4505–4515. doi:10.1109/TCSI.2023.3306347

[7] Wenbo Guo and Shuguo Li. 2024. Split-Radix Based Compact Hardware Architecture for CRYSTALS-Kyber. *IEEE Trans. Comput.* 73, 1 (2024), 97–108. doi:10.1109/TC.2023.3320040

[8] Malik Imran, Safiullah Khan, Zain Ul Abideen, Ciara Rafferty, Ayesha Khalid, Muhammad Rashid, and Máire O'Neill. 2025. RejSCore: Rejection Sampling Core for Multivariate-based Public key Cryptography. In *2025 Asian Hardware Oriented Security and Trust Symposium (AsianHOST)*. 1–6. doi:10.1109/AsianHOST68425.2025.11370348

[9] Hyunseon Kim, Haesung Jung, Ardianto Satriawan, and Hanho Lee. 2024. A Configurable ML-KEM/Kyber Key-Encapsulation Hardware Accelerator Architecture. *IEEE Transactions on Circuits and Systems II: Express Briefs* 71, 11 (2024), 4678–4682. doi:10.1109/TCSII.2024.3442228

[10] Neal Koblitz. 1987. Elliptic curve cryptosystems. *Mathematics of computation* 48, 177 (1987), 203–209.

[11] Victor S Miller. 1985. Use of elliptic curves in cryptography. In *Conference on the theory and application of cryptographic techniques*. Springer, 417–426.

[12] Jianan Mu, Yi Ren, Wen Wang, Yizhong Hu, Shuai Chen, Chip-Hong Chang, Junfeng Fan, Jing Ye, Yuan Cao, Huawei Li, and Xiaowei Li. 2023. Scalable and Conflict-Free NTT Hardware Accelerator Design: Methodology, Proof, and Implementation. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 5 (2023), 1504–1517. doi:10.1109/TCAD.2022.3205552

[13] National Institute of Standards and Technology (NIST). 2024. Module-Lattice-Based Key-Encapsulation Mechanism Standard. doi:10.6028/NIST.FIPS.203

[14] Ziying Ni, Ayesha Khalid, Dur-e-Shahwar Kundi, Máire O'Neill, and Weiqiang Liu. 2023. HPKA: A High-Performance CRYSTALS-Kyber Accelerator Exploring Efficient Pipelining. *IEEE Trans. Comput.* 72, 12 (2023), 3340–3353. doi:10.1109/TC.2023.3296899

[15] R. L. Rivest, A. Shamir, and L. Adleman. 1978. A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM* 21, 2 (Feb. 1978), 120–126. doi:10.1145/359340.359342

[16] Peter W. Shor. 1999. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM Rev.* 41, 2 (1999), 303–332. doi:10.1137/S0036144598347011

[17] Bardia Taghavi, Reza Azarderakhsh, and Mehran Mozaffari Kermani. 2025. ParallelNTT: Maximizing Performance of Forward and Inverse NTT on FPGA for ML-DSA and ML-KEM. In *Proceedings of the Great Lakes Symposium on VLSI 2025 (GLSVLSI '25)*. Association for Computing Machinery, New York, NY, USA, 372–378. doi:10.1145/3716368.3735214

[18] Yufei Xing and Shuguo Li. 2021. A Compact Hardware Implementation of CCA-Secure Key Exchange Mechanism CRYSTALS-KYBER on FPGA. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2021, 2 (Feb. 2021), 328–356. doi:10.46586/tches.v2021.i2.328-356